

Module 3:

Constructs Deep Dive + Mini-Lab

Learning Agenda

- By the end of this module, learners will:
 - Understand the differences between **L1, L2, and L3 constructs**
 - Read CDK documentation effectively
 - Connect constructs together using helper methods (e.g., grantWrite)
 - Use **escape hatches** to modify properties not exposed by L2
 - Import existing AWS resources into CDK
 - Build and deploy a small app using L2 constructs with permissions and an escape hatch change

What Are CDK Constructs?

- Fundamental building block in CDK
- Represent AWS resources (S3 buckets, Lambdas, VPCs, APIs, etc.)
- Three levels: **L1, L2, L3**
- Preferred level: **L2**

L2 Constructs (Well-Curated AWS Abstractions)

- Opinionated defaults
- Safer and easier than L1
- Use L2 whenever possible
- Example: `new s3.Bucket(this, 'MyBucket')`

Practical Example 1 — Create a versioned S3 Bucket

```
const uploadsBucket = new s3.Bucket(this, 'UploadsBucket', {  
  versioned: true,  
  blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,  
  encryption: s3.BucketEncryption.S3_MANAGED,  
});
```

Practical Example 2 — Create a Lambda with inline code

```
const uploaderFn = new lambda.Function(this, 'UploaderFn', {  
  runtime: lambda.Runtime.NODEJS_18_X,  
  handler: 'index.handler',  
  code: lambda.Code.fromInline(`  
    exports.handler = async (event) => {  
      console.log("File uploaded event:", event);  
    };  
  `),  
});
```

Practical Example 3 — Create a DynamoDB table

```
const table = new dynamodb.Table(this, 'FilesTable', {  
  tableName: 'FileMetadata',  
  billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,  
  partitionKey: { name: 'fileId', type: dynamodb.AttributeType.STRING },  
  removalPolicy: RemovalPolicy.DESTROY,  
});
```

Wiring Constructs Together + Helper Methods

- Many L2 constructs include helper methods
- Examples:
 - `bucket.grantRead(lambdaFn)`
 - `table.grantWriteData(lambdaFn)`
 - `api.addRoutes()`

L1 Constructs + Escape Hatches

- **L1 = CloudFormation**
- You use L1 when:
 - AWS releases a new feature not yet in L2
 - You need a property L2 doesn't expose
- Escape hatches: modify generated CloudFormation

Example: Create a bucket using L1 (raw CloudFormation)

```
const rawBucket = new s3.CfnBucket(this, 'RawBucket', {  
  bucketName: 'my-raw-bucket-' + Date.now(),  
  versioningConfiguration: { status: 'Enabled' },  
});
```

Escape Hatch Example – Modify an L2 Bucket's raw CFN

- **Example: Add server access logs (not exposed by L2 initially)**

```
const bucket = new s3.Bucket(this, 'MainBucket');
```

```
const cfnBucket = bucket.node.defaultChild as s3.CfnBucket;  
cfnBucket.addPropertyOverride('LoggingConfiguration', {  
  DestinationBucketName: 'my-log-bucket',  
  LogFilePrefix: 'app/',  
});
```

Example: Add IAM policy via escape hatch

```
const role = uploaderFn.role!;  
role.addToPrincipalPolicy(new iam.PolicyStatement({  
  actions: ['s3:ListAllMyBuckets'],  
  resources: ['*'],  
}));
```

L3 Patterns & Importing Existing Resources

- L3 = “patterns”
- Provided by:
 - AWS Solutions Constructs
 - Internal enterprise libraries
- Import existing resources:
 - `s3.Bucket.fromBucketName()`
 - `lambda.Function.fromFunctionArn()`

Example – S3 → Lambda Pattern

```
new solutionspatterns.S3ToLambda(this, 'S3TriggerLambda', {  
  lambdaFunctionProps: { runtime: lambda.Runtime.NODEJS_18_X },  
});
```

This:

- Creates S3 bucket
- Creates Lambda
- Connects event notification
- Handles IAM permissions

Example – Lambda → DynamoDB Pattern

```
new solutionspatterns.LambdaToDynamoDB(this, 'LambdaToDDB', {  
  lambdaFunctionProps: { runtime: lambda.Runtime.NODEJS_18_X },  
  dynamoTableProps: {  
    partitionKey: { name: 'id', type: dynamodb.AttributeType.STRING }  
  }  
});
```

Questions?